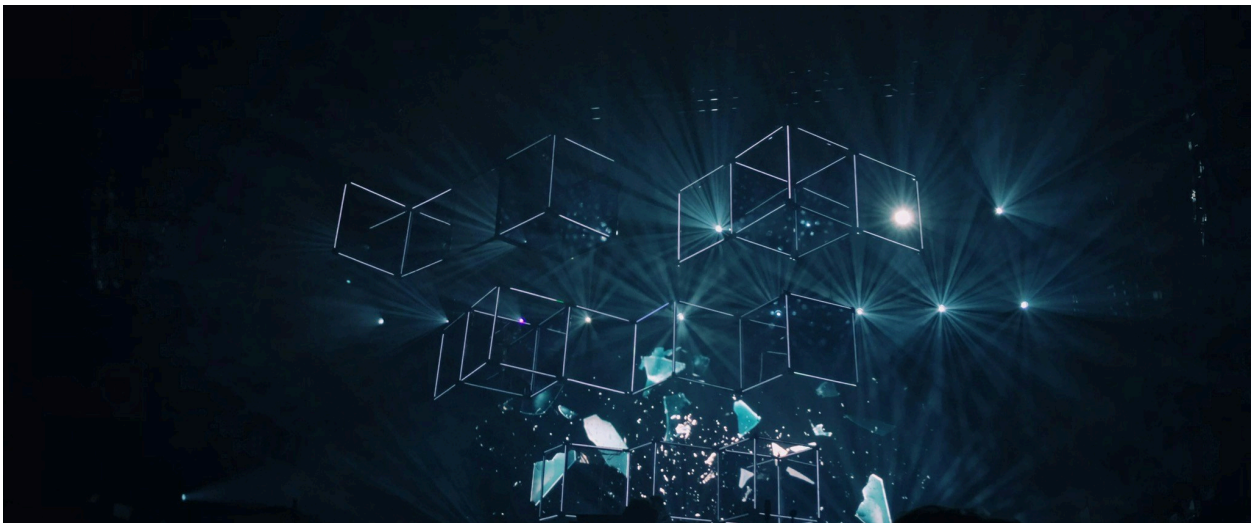

DQL (Data Query Language)

Kranti Patil
Fusion Software institute

What is DQL?

Data Query Language (DQL) is a subset of SQL (Structured Query Language) used to retrieve data from databases. DQL statements are primarily concerned with querying and fetching data without modifying the database structure or content.

Basic SELECT Statement

The SELECT statement is the foundation of DQL and is used to retrieve data from one or more tables.

Basic Syntax

```
SELECT column1, column2, ...
FROM table_name;
```

SELECT ALL Columns

To retrieve all columns from a table, use the asterisk (*) wildcard:

```
SELECT * FROM employees;
```

SELECT Specific Columns

Specify only the columns you need for better performance:

```
SELECT first_name, last_name, salary
FROM employees;
```

WHERE Clause

The WHERE clause filters records based on specified conditions:

```
SELECT * FROM employees
WHERE salary > 50000;
```

Key Points:

- Filters rows before they are returned
- Can use multiple conditions with AND/OR
- Essential for data filtering
- Improves query performance by reducing result set

Examples:

```
-- Single condition
SELECT * FROM products WHERE price > 100;

-- Multiple conditions
SELECT * FROM employees
WHERE department = 'IT' AND salary > 60000;
```

GROUP BY Clause

Groups rows with the same values in specified columns:

```
SELECT department, COUNT(*) as employee_count
FROM employees
GROUP BY department;
```

Important Rules:

- Every column in SELECT must either be in GROUP BY or use an aggregate function
- Used with aggregate functions (COUNT, SUM, AVG, MAX, MIN)
- Groups are processed before ORDER BY

Common Aggregate Functions:

- `COUNT()` - Counts rows
- `SUM()` - Sums numeric values
- `AVG()` - Calculates average
- `MAX()` - Finds maximum value
- `MIN()` - Finds minimum value

DISTINCT Keyword

Removes duplicate rows from the result set:

```
SELECT DISTINCT department FROM employees;
```

Usage Examples:

```
-- Single column
SELECT DISTINCT city FROM customers;
```

```
-- Multiple columns
SELECT DISTINCT department, job_title FROM employees;
```

Key Points:

- Eliminates duplicate rows
- Can be used with multiple columns
- May impact performance on large datasets

Aliases (AS Keyword)

Provides temporary names for columns or tables:

Column Aliases

```
SELECT
    first_name AS "First Name",
    last_name AS "Last Name",
    salary * 12 AS "Annual Salary"
FROM employees;
```

Table Aliases

```
SELECT e.first_name, e.last_name, d.department_name
FROM employees AS e
JOIN departments AS d ON e.dept_id = d.id;
```

Benefits:

- Makes output more readable
- Shortens long table names
- Required for calculated columns
- Essential for self-joins

ORDER BY Clause

Sorts the result set by one or more columns:

```
SELECT * FROM employees
ORDER BY last_name ASC, first_name DESC;
```

Sorting Options:

- `ASC` - Ascending order (default)
- `DESC` - Descending order
- Multiple columns supported
- Can sort by column position number

Examples:

```
-- Single column ascending
SELECT * FROM products ORDER BY price;

-- Multiple columns
SELECT * FROM employees
ORDER BY department ASC, salary DESC;
```

Arithmetic Operators

Perform mathematical operations on numeric data:

Operator	Description	Example
+	Addition	`salary + bonus`
-	Subtraction	`price - discount`
*	Multiplication	`quantity * unit_price`
/	Division	`total_amount / quantity`
%	Modulo	`employee_id % 2`

Examples:

```
-- Calculate annual salary
SELECT first_name, salary, salary * 12 AS annual_salary
FROM employees;

-- Calculate profit margin
SELECT product_name,
       selling_price - cost_price AS profit,
       ((selling_price - cost_price) / selling_price) * 100 AS profit_percentage
FROM products;
```

Relational Operators

Compare values in WHERE clauses:

Operator	Description	Example
=	Equal to	`salary = 50000`
!= or <>	Not equal to	`department != 'HR'`
>	Greater than	`age > 25`
<	Less than	`price < 100`
>=	Greater than or equal	`salary >= 40000`
<=	Less than or equal	`experience <= 5`

Examples:

```
-- Equal comparison
SELECT * FROM employees WHERE department = 'IT';

-- Range comparison
SELECT * FROM products WHERE price >= 50 AND price <= 200;

-- Not equal
SELECT * FROM customers WHERE city <> 'New York';
```

Logical Operators

Combine multiple conditions:

AND Operator

```
SELECT * FROM employees
WHERE department = 'IT' AND salary > 60000;
```

OR Operator

```
SELECT * FROM employees
WHERE department = 'IT' OR department = 'Finance';
```

NOT Operator

```
SELECT * FROM employees
WHERE NOT department = 'HR';
```

Precedence:

1. NOT (highest)
2. AND
3. OR (lowest)

Use parentheses for clarity:

```
SELECT * FROM employees
WHERE (department = 'IT' OR department = 'Finance')
  AND salary > 50000;
```

LIKE Operator

Used for pattern matching in text fields:

Wildcards

- `%` - Matches zero or more characters
- `_` - Matches exactly one character

Examples:

```
-- Names starting with 'A'
SELECT * FROM employees WHERE first_name LIKE 'A%';

-- Names ending with 'son'
SELECT * FROM employees WHERE last_name LIKE '%son';

-- Names containing 'john'
SELECT * FROM employees WHERE first_name LIKE '%john%';

-- Exactly 5 characters
SELECT * FROM products WHERE product_code LIKE '____';

-- Second character is 'a'
SELECT * FROM employees WHERE first_name LIKE '_a%';
```

IN Operator

Checks if a value exists in a list of values:

```
SELECT * FROM employees
WHERE department IN ('IT', 'Finance', 'Marketing');
```

Advantages over OR:

- More concise syntax
- Better performance in some cases
- Easier to read and maintain

With Subqueries:

```
SELECT * FROM employees
WHERE department_id IN (
    SELECT id FROM departments
    WHERE location = 'New York'
);
```

NOT IN:

```
SELECT * FROM employees
WHERE department NOT IN ('HR', 'Admin');
```

BETWEEN Operator

Checks if a value falls within a range:

```
SELECT * FROM employees
WHERE salary BETWEEN 40000 AND 80000;
```

Key Points:

- Inclusive of boundary values
- Works with numbers, dates, and text
- Equivalent to `>= AND <=`

Examples:

```
-- Numeric range
SELECT * FROM products WHERE price BETWEEN 10 AND 50;

-- Date range
SELECT * FROM orders
WHERE order_date BETWEEN '2023-01-01' AND '2023-12-31';

-- Text range (alphabetical)
SELECT * FROM employees
```

```
WHERE last_name BETWEEN 'A' AND 'M';

-- NOT BETWEEN
SELECT * FROM employees
WHERE salary NOT BETWEEN 30000 AND 70000;
```

NULL Values

```
-- Check for NULL values
SELECT * FROM employees WHERE phone IS NULL;

-- Check for non-NULL values
SELECT * FROM employees WHERE phone IS NOT NULL;
```

LIMIT/TOP Clause

```
-- MySQL/PostgreSQL
SELECT * FROM employees ORDER BY salary DESC LIMIT 10;

-- SQL Server
SELECT TOP 10 * FROM employees ORDER BY salary DESC;
```